SafeDocs

Secure PDF Management System

CS 432 – Databases
Assignment 3: Transaction Management, Concurrency Control, and
ACID Validation on Application and B+ Tree Database

Track 1 — Statement 6

Name	Roll Number
Sujal Patel	22110261
Siddharth Doshi	22110250
Viraj Vekaria	22110287
Nishit Mistry	22110172
Rutvi Shah	22110227

Indian Institute of Technology, Gandhinagar

Semester II (2025–2026)

Instructor: Dr. Yogesh K. Meena

GitHub Repository: https://github.com/SujalAVMP/SafeDocs

Video Demonstration: https://youtu.be/aaZb44vIZuA

April 2026

Contents

1 Introduction **2**
1.1 Assignment Objectives . . . 2
1.2 Verification Strategy . . . 2

2 Module A: Advanced Transaction Engine and Crash Recovery **2**
2.1 Introduction . . . 2
2.2 Design Overview . . . 3
2.3 Transaction Execution Model . . . 3
2.4 Consistency Between Records and the B+ Tree . . . 3
2.5 Durable Commit and Recovery . . . 4
2.6 ACID Validation Scenarios . . . 4
2.7 Observations . . . 5
2.8 Limitations . . . 5
2.9 Module A: Conclusion . . . 6

3 Module B: High-Concurrency API Load Testing and Failure Simulation **7**
3.1 Introduction . . . 7
3.2 Runtime Setup . . . 7
3.3 Failure Handling in the Application Layer . . . 7
3.4 Stress Harness Design . . . 7
3.5 Stress Test Results . . . 8
3.6 Visual Assignment 3 Test Console . . . 8
3.7 Role-Based Access on the Visual Console . . . 9
3.8 Observations . . . 9
3.9 Limitations . . . 10
3.10 Module B: Conclusion . . . 10

4 Coverage of Assignment Requirements **11**

5 Tools and External Sources **11**

6 Team Member Contributions **12**

7 Conclusion **12**

1 Introduction

Assignment 3 extends SafeDocs from a functional course project into a system that explicitly demonstrates correctness under failures and concurrent access. The assignment is divided into two complementary modules:

- **Module A:** Advanced transaction management, rollback, durability, and crash recovery inside the custom B+ Tree-backed database engine.
- **Module B:** Multi-user behaviour, rollback testing, race-condition validation, and stress testing of the Flask + MySQL application layer.

The objective is not only to make operations work in the normal case, but to show that the system remains correct when requests fail midway, when multiple users act at the same time, and when the runtime is restarted after a partially completed execution.

1.1 Assignment Objectives

The assignment requires the following guarantees:

- **Atomicity:** Every operation must either complete fully or leave no partial effect.
- **Consistency:** Invalid operations must be rejected without leaving the system in an incorrect state.
- **Isolation:** Concurrent users must not interfere in a way that produces oversell, double-delete, or other incorrect results.
- **Durability:** Once a transaction commits, the committed state must survive restart or failure.
- **Recovery:** Incomplete or interrupted operations must be handled safely after restart.

1.2 Verification Strategy

We verified the final implementation in two ways:

1. By running the Assignment 3 scripts directly from the command line:
   - `python Module_A\assignment3_demo.py`
   - `python stress\assignment3_stress.py`
2. By integrating a visual Assignment 3 console into Module B at `/assignment3/tests`, which runs the same scripts as subprocesses and shows live logs, scenario states, and final pass/fail summaries.

2 Module A: Advanced Transaction Engine and Crash Recovery

2.1 Introduction

Module A builds on the B+ Tree and typed relational abstractions from Assignment 2. The goal in Assignment 3 is to make this engine transaction-safe. Instead of applying updates directly to the live database image, we introduce an explicit transaction coordination layer that stages work, commits it durably, and recovers cleanly after failures.

This module is implemented primarily in:

- `Module_A/database/transaction_manager.py`
- `Module_A/database/persistence.py`
- `Module_A/assignment3_demo.py`

2.2 Design Overview

The transaction layer is centered on three components:

- **TransactionCoordinator** – manages the live committed database state, begins transactions, commits staged work, rolls back failed work, and triggers recovery.
- **Transaction** – a staged transactional view over a cloned `DatabaseManager`.
- **SnapshotJournalStore** – persists transaction lifecycle events and the latest committed state using a checkpoint snapshot and a JSONL journal.

The key design decision is that every transaction operates on a *cloned logical image* of the database. The live state is not modified during execution. This prevents partially applied updates from leaking into the committed system state.

2.3 Transaction Execution Model

The transaction flow is:

1. `BEGIN` acquires a global re-entrant lock and records a `BEGIN` entry in the journal.
2. The current `DatabaseManager` is cloned using serialization and reconstruction.
3. All inserts, updates, and deletes occur only on the staged copy.
4. If a constraint or runtime error occurs before commit, the staged copy is discarded and `ROLLBACK` is recorded.
5. If the transaction commits, the full committed state is first written to the durable journal.
6. Only after that durable write succeeds is the staged manager promoted to the live manager.
7. A checkpoint snapshot is then written to disk.

This staged execution model provides the assignment properties in a direct way:

- **Atomicity** comes from all-or-nothing publication of the staged state.
- **Consistency** comes from explicit transaction-level checks (`require(...)`) and typed table validation.
- **Isolation** comes from serialized execution via a single global lock.
- **Durability** comes from journal-first commit followed by checkpointing.

2.4 Consistency Between Records and the B+ Tree

The `Table` abstraction uses the B+ Tree as the authoritative in-memory storage structure. The table's persisted form is derived from B+ Tree contents, and `DatabaseManager.clone()` creates a deep logical copy by calling `to_dict()` and `from_dict()` across the database hierarchy.

This means:

- the staged transaction image is internally consistent,
- the committed state is written as a complete database image,
- recovery reconstructs both records and indexes together,
- logical rows and B+ Tree entries cannot diverge due to a partial commit.

This satisfies one of the most important requirements of the assignment: correctness must hold not only for the logical database view, but also for the underlying B+ Tree-backed storage structure.

2.5 Durable Commit and Recovery

Module A stores two persistence artifacts for each Assignment 3 run:

- `snapshot.json` – the latest checkpointed committed database image
- `journal.jsonl` – an append-only log of `BEGIN`, `COMMIT`, and `ROLLBACK` entries

Recovery follows a simple durable rule:

1. load the last checkpoint snapshot if it exists,
2. scan the journal,
3. identify the latest durable committed state,
4. ignore transactions that began but never committed,
5. reconstruct the live `DatabaseManager` from the latest durable committed payload.

This makes two critical failure cases easy to validate:

- an incomplete transaction with `BEGIN` but no `COMMIT` is discarded after restart,
- a transaction that journaled `COMMIT` but crashed before snapshot completion is still recovered correctly from the journal.

2.6 ACID Validation Scenarios

The script `Module_A/assignment3_demo.py` runs a three-table scenario over `users`, `products`, and `orders`. Each scenario prints step-by-step logs and then verifies correctness with assertions.

Table 1 summarizes the latest successful verification run.

Table 1: Module A Assignment 3 verification scenarios

Scenario	**Test Method**	**Observed Outcome**
Atomicity	Injected failure after updating the user balance in order `5001`	User balance remained `1000.0`, product stock remained `5`, and no order row was persisted.
Consistency	Submitted an invalid purchase with quantity `99`	The transaction was rejected with `insufficient balance`; all tables remained unchanged.
Isolation	Two concurrent buyers competed for the last available unit	Exactly one transaction committed, one rolled back, final stock became `0`, and exactly one order row remained.
Durability	Committed order `7001` and restarted the coordinator	Recovered user balance `500.0`, recovered stock `3`, and committed order row present.
Recovery – Incomplete Transaction	Began a transaction, wrote staged changes, and restarted without commit	Original balance `1000.0` and stock `5` were restored; incomplete order absent.
Recovery – Journal Replay	Simulated crash after durable journaled commit but before snapshot completion	Restart recovered balance `750.0`, stock `4`, and the committed order row correctly.

2.7 Observations

The Module A implementation is deliberately simple and therefore easy to reason about. Its main strengths are:

- commit publication is all-or-nothing,
- rollback is explicit and observable,
- recovery behavior is deterministic,
- table state and B+ Tree state remain aligned throughout transaction execution.

The latest verification run passed all six scenarios successfully, confirming that the engine currently satisfies the assignment's correctness requirements.

2.8 Limitations

The current Module A design is intentionally conservative:

- Isolation is achieved by serializing all transactions with a single global lock, not by fine-grained locking or MVCC.
- The journal stores the full committed database image, which is easy to recover but not space-efficient.

- Recovery is not ARIES-style undo/redo logging; it is a latest-committed-state replay model.
- The demonstration workload is small and deterministic, which favors clarity over throughput realism.

2.9 Module A: Conclusion

Module A now demonstrates transaction correctness inside the custom storage engine itself. The staged transaction model, durable journal-first commit path, checkpoint snapshots, and restart recovery together provide a clean implementation of atomicity, consistency, isolation, durability, and recovery for the assignment setting.

3 Module B: High-Concurrency API Load Testing and Failure Simulation

3.1 Introduction

Module B evaluates the application layer under failures and concurrent use. The system uses Flask for the API and web UI, MySQL for persistent storage, and JWT-based session authentication. Assignment 3 extends this module with:

- concurrent stress testing,
- rollback verification under API failure,
- race-condition validation on shared resources,
- a visual Assignment 3 test console that runs the verification scripts and displays live output.

3.2 Runtime Setup

For Assignment 3 verification, Module B runs in a Linux-style Docker environment:

- `safedocs-mysql` – MySQL 8.0 container
- `safedocs-app` – Flask application container

The stress harness in `Module_B/stress/assignment3_stress.py` targets the live API at `http://127.0.0.1:5000`.

3.3 Failure Handling in the Application Layer

The Module B Assignment 3 checks focus on three concrete risks:

Rollback integrity. The route `POST /api/members` inserts into `Member` and then into `UserLogin`. Reusing the existing username `admin` forces the second insert to fail. The route must rollback the first insert so that no partial member row remains.

Race-condition safety. The route `DELETE /api/documents/<id>` performs a soft delete by setting `IsActive = FALSE`. Multiple concurrent delete requests against the same document must not produce an inconsistent final state.

Load stability. The application must continue serving concurrent authenticated API requests without 5xx failures or broken state transitions.

Because MySQL access uses `autocommit=False`, each request can explicitly commit only after all required steps complete successfully. This is essential for the rollback-integrity scenario.

3.4 Stress Harness Design

The stress harness performs three named scenarios:

- `rollback_integrity`
- `delete_race`

- `mixed_load`

The implementation uses `ThreadPoolExecutor` to simulate multiple concurrent clients and checks both correctness and throughput-related behavior:

- a failure-oriented write scenario for rollback correctness,
- a shared-resource race scenario for delete safety,
- a read-heavy mixed-load scenario for repeated API usage.

3.5 Stress Test Results

Table 2 summarizes the latest successful run of the Module B stress suite.

Table 2: Module B Assignment 3 verification results

Scenario	Workload	Observed Result
`rollback integrity`	Duplicate-username member creation using `admin`	Returned HTTP `409`, post-check count stayed `0`, and no partial member row was created.
`delete race`	`8` concurrent delete requests against the same temporary document	Exactly one request returned `200`, seven returned `404`, and the final `GET` also returned `404`.
`mixed load`	`240` total requests across `24` workers	All `240` requests returned `200`; average latency `120.21 ms`, median `101.58 ms`, p95 `214.11 ms`, max `272.59 ms`, wall-clock `1248.42 ms`.

The combined harness result was:

- **Overall result: PASS**

3.6 Visual Assignment 3 Test Console

To improve demonstration quality, we added a visual Assignment 3 console to Module B at:

`/assignment3/tests`

This page does not reimplement the test logic. Instead, it launches the existing Assignment 3 scripts as subprocesses and streams their live output to the browser. The console provides:

- separate Module A and Module B panels,
- a combined "Run All" orchestration option,
- live log panels,
- per-scenario status badges,

- pass/fail summaries,
- busy-state protection against overlapping runs,
- role-based access control for Admin and Auditor users.

For Module A web-triggered runs, we introduced the environment variable `SAFEDOCS_A3_RUNTIME_DIR` so that runtime artifacts are written to a writable temporary path inside the container while preserving the original CLI behavior.

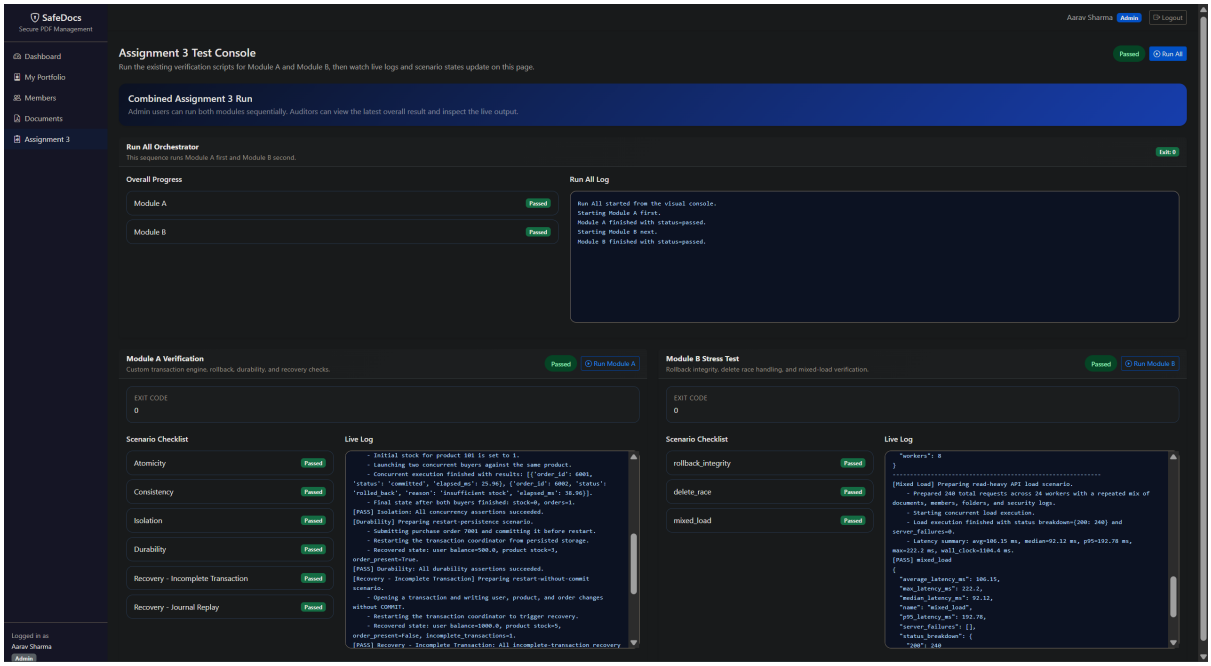

Figure 1: Assignment 3 visual test console integrated into Module B. The page runs the existing verification scripts and shows live scenario status and logs.

3.7 Role-Based Access on the Visual Console

The visual console also demonstrates application-level access control:

- **Admin** can view the page, run Module A, run Module B, and run all checks together.
- **Auditor** can view the page and run Module A only.
- **Auditor** can still inspect the latest Module B results and logs, but Module B execution controls remain disabled.

This makes the console suitable for both live demonstration and controlled review.

3.8 Observations

The Module B results show that:

- failed multi-step member creation rolls back cleanly,
- the document delete path remains safe under concurrent contention,
- the application handles a moderately heavy local read workload without server failures,
- the API and MySQL stack remain stable under repeated authenticated usage.

The delete-race scenario is particularly important because it validates correctness under simultaneous destructive operations on the same resource.

3.9 Limitations

The current Module B validation has the following scope limits:

- It is a local benchmark rather than a distributed production benchmark.
- The mixed-load scenario is intentionally read-heavy rather than write-heavy.
- The visual console stores only the latest results in memory, so its state is lost on application restart.
- The module validates correctness through application logic and MySQL transaction handling; it does not implement custom low-level concurrency control in the storage engine.

3.10 Module B: Conclusion

Module B demonstrates that the application layer can rollback failed multi-step requests, handle concurrent races safely, and remain stable under repeated authenticated load. The addition of the visual Assignment 3 console also makes the verification process clearer and more suitable for video demonstration.

4 Coverage of Assignment Requirements

Table 3 maps the assignment's evaluation criteria to concrete evidence from this submission.

Table 3: Assignment requirement coverage

Evaluation Criterion	**Evidence in This Submission**
Correctness of transaction behaviour	Module A uses staged transactions, rollback, journal-first commit, and six passing verification scenarios.
Proper handling of failures	Module A demonstrates incomplete-transaction recovery and journal replay; Module B demonstrates rollback integrity under API failure.
Multi-user safety and isolation	Module A serializes concurrent buyers correctly; Module B validates concurrent delete safety using the `delete_race` scenario.
Consistency between database and B+ Tree	`Table` uses the B+ Tree as the authoritative data structure, and persisted state is cloned and recovered as a complete database image.
System robustness under load	Module B completed `240/240` successful requests with no server failures in the latest mixed-load run.
Clarity of explanation	Command-line logs, deterministic assertions, tables of results, and the visual Assignment 3 console support transparent demonstration.

5 Tools and External Sources

As required by the submission guidelines, we disclose the tools and external resources used:

- **Programming language:** Python 3.12
- **Web framework:** Flask
- **Database:** MySQL 8.0
- **Authentication:** PyJWT
- **Database driver:** PyMySQL
- **Container runtime:** Docker
- **Frontend:** Bootstrap 5 and Jinja2
- **Visualization support:** Graphviz (from earlier module visualizations), Matplotlib, Jupyter Notebook
- **Reference material:** Previous project reports (`Report_A1.pdf`, `Report_A2.pdf`), Python documentation, Flask documentation, and MySQL documentation

6 Team Member Contributions

Name	Contribution
Sujal Patel (22110261)	Module B integration, API-side verification workflow, demo support
Siddharth Doshi (22110250)	Module A transaction coordination, rollback, journaling, recovery logic
Viraj Vekaria (22110287)	Assignment 3 validation flow, report drafting, demo refinement
Nishit Mistry (22110172)	Docker/Linux execution setup, MySQL verification, stress-test support
Rutvi Shah (22110227)	Test review, documentation support, video coordination

7 Conclusion

Assignment 3 makes SafeDocs robust under both failure and concurrency.

Module A shows that the custom B+ Tree-backed engine can:

- stage transactions safely,
- rollback partial work,
- recover correctly after restart,
- preserve committed state,
- maintain consistency between records and indexes.

Module B shows that the application layer can:

- rollback failed multi-step requests,
- handle concurrent races without corrupting state,
- remain stable under sustained authenticated load,
- expose the entire verification flow through a visual test console.

Taken together, these results satisfy the core objective of the assignment: a system that works correctly, handles failures safely, and supports multi-user operation without breaking correctness.